23. Merge k Sorted Lists Hard

Acceptance: 58.6% | LeetCode

You are given an array of `k` linked-lists `lists`, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

```
**Input:** lists = [[1,4,5],[1,3,4],[2,6]]
**Output:** [1,1,2,3,4,4,5,6]
**Explanation:** The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
merging them into one sorted linked list:
1->1->2->3->4->4->5->6
```

Example 2:

```
**Input:** lists = []
**Output:** []
```

Example 3:

```
**Input:** lists = [[]]
**Output:** []
```

Constraints:

- `k == lists.length`
- `0 <= k <= 104`
- `0 <= lists[i].length <= 500`
- `-104 <= lists[i][j] <= 104`
- `lists[i]` is sorted in **ascending order**.
- The sum of `lists[i].length` will not exceed `104`.

Claude Opus Solution

Merge k Sorted Lists

1. Intuition

The key insight: at any moment, the **next smallest node** in the final merged list must be the **smallest among the current heads** of all `k` lists. Once we pick the smallest head, we advance that list by one node and repeat.

We need a data structure that efficiently gives us the minimum among `k` candidates and lets us insert new candidates. A **min-heap (priority queue)** is the perfect fit — it gives us the minimum in `O(log k)` time.

This is the classic **k-way merge** pattern. Instead of merging all lists at once (slow), we always track only `k` candidates (one head per list) in the heap.

---

2. Approach (Pseudocode)

```
1. Handle edge cases: empty input → return None
2. Push the head of every non-empty list into a min-heap
   - Heap key = node.val (smallest value floats to top)
3. Create a dummy node to simplify list construction
4. While heap is not empty:
   a. Pop the node with the smallest value
   b. Attach it to the result list
   c. If the popped node has a next node, push next into the heap
5. Return dummy.next (the merged list)
```

Why `id(node)` as a tiebreaker? Python's heap compares tuples element by element. If two nodes have equal `val`, Python would try to compare the `ListNode` objects themselves — which raises a `TypeError` (no `<` defined). Adding a unique `id(node)` breaks ties before the comparison ever reaches the node object.

---

3. Python Solution

```python
from heapq import heappush, heappop
from typing import List, Optional


class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

```python
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        # Edge case: no lists at all
        if not lists:
            return None

        # Min-heap will hold tuples: (value, unique_tiebreaker, node)
        heap = []

        # Step 1: Seed the heap with the head of each non-empty list.
        # This gives us the initial set of "candidates" — one per list.
        for node in lists:
            if node is not None:                    # skip empty lists ([])
                # id(node) is a unique int, ensuring heapq never has to
                # compare two ListNode objects (which would crash).
                heappush(heap, (node.val, id(node), node))

        # Step 2: Use a dummy head so we don't special-case the first node.
        # 'current' is the tail pointer where we append the next smallest node.
        dummy = current = ListNode()

        # Step 3: Repeatedly extract the smallest node and rebuild the list.
        while heap:
            # Pop the node with the minimum value across all current heads.
            _, _, node = heappop(heap)

            # Append it to our growing result list.
            # This single line both links (current.next = node)
            # and advances the tail pointer (current = node).
            current.next = current = node

            # If this list still has more nodes, push the next one as a
            # new candidate. This keeps the heap size <= k.
            if node.next:
                nxt = node.next
                heappush(heap, (nxt.val, id(nxt), nxt))

        # dummy.next points to the actual head of the merged list.
        return dummy.next
```

---

4. Complexity Analysis

Let **n** = total number of nodes across all lists, **k** = number of lists.

Time Complexity: `O(n log k)` - Each of the `n` nodes is pushed and popped from the heap exactly once. - The heap never exceeds size `k` (we only keep one candidate per list at a time), so each push/pop costs `O(log k)`. - Total: `n × O(log k) = O(n log k)`.

Space Complexity: `O(k)` - The heap holds at most `k` nodes simultaneously. - We reuse the existing nodes (no new node allocation), so output space isn't counted as extra.

---

5. Alternative Approaches

Approach	Time	Space	Notes
Min-Heap (above)	`O(n log k)`	`O(k)`	Clean, optimal.
Divide & Conquer (merge pairs)	`O(n log k)`	`O(log k)` recursion	Merge lists in pairs like merge-sort; same optimal time, lower aux space.
Brute force: merge one-by-one	`O(n·k)`	`O(1)`	Merge list[0] with list[1], then result with list[2]... Slow when k is large.
Collect all values, sort	`O(n log n)`	`O(n)`	Simple but ignores the "already sorted" property; worse time.

Divide & Conquer Sketch (the other optimal solution)

```python
class Solution:
    def mergeKLists(self, lists):
        if not lists:
            return None
```

```
        # Repeatedly merge lists in pairs until one remains.
        while len(lists) > 1:
            merged = []
            # Pair up lists: (0,1), (2,3), ...
            for i in range(0, len(lists), 2):
                l1 = lists[i]
                l2 = lists[i + 1] if i + 1 < len(lists) else None
                merged.append(self.mergeTwo(l1, l2))
            lists = merged              # halve the count each round
        return lists[0]

    def mergeTwo(self, a, b):
        # Standard merge of two sorted lists (like merge sort's merge step).
        dummy = tail = ListNode()
        while a and b:
            if a.val <= b.val:
                tail.next, a = a, a.next
            else:
                tail.next, b = b, b.next
            tail = tail.next
        tail.next = a or b              # attach remaining tail
        return dummy.next
```

Why same complexity? There are `log k` merge rounds, and each round processes all `n` nodes once → `O(n log k)`. This avoids the heap's overhead and uses only `O(log k)` recursion/auxiliary space, making it slightly more space-efficient in practice.

Codex Solution

1. Intuition

We have `k` sorted linked lists. At any moment, the smallest remaining node among all lists must be one of the current heads of those lists.

So the key pattern is:

> Keep track of the smallest current head among all lists using a **min-heap**.

A min-heap lets us efficiently get the smallest available node in `O(log k)` time, where `k` is the number of lists.

After we take the smallest node from the heap, we add its `next` node to the heap, because that node now becomes the new candidate from that same list.

---

2. Approach

Pseudocode

```
mergeKLists(lists):

    create an empty min_heap

    for each linked list head in lists:
        if head is not None:
            push (head.val, unique_id, head) into min_heap

    create dummy node
    current = dummy

    while min_heap is not empty:
        pop the node with the smallest value

        attach this node to the merged list
        current.next = node
        current = current.next

        if node.next exists:
            push (node.next.val, unique_id, node.next) into min_heap

    return dummy.next
```

Why use `unique_id`?

In Python, if two heap entries have the same value, Python tries to compare the next element in the tuple.

For example:

```
(val, node)
```

If two `val`s are equal, Python tries to compare `node`, but `ListNode` objects are not directly comparable.

So we store:

```
(val, unique_id, node)
```

The `unique_id` breaks ties safely.

---

3. Python Solution

```python
from typing import List, Optional
import heapq


# Definition for singly-linked list.
# LeetCode already provides this class.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        # Min-heap storing tuples:
        # (node value, unique tie-breaker, node reference)
        heap = []

        # Unique counter to avoid comparing ListNode objects
        # when two nodes have the same value.
        unique_id = 0

        # Add the head of each non-empty linked list to the heap.
        # At the start, only the heads can possibly be the smallest nodes.
        for head in lists:
            if head is not None:
                heapq.heappush(heap, (head.val, unique_id, head))
                unique_id += 1

        # Dummy node simplifies building the result list.
        # dummy.next will point to the real head of the merged list.
        dummy = ListNode()
        current = dummy

        # Continue until all nodes have been processed.
        while heap:
            # Extract the node with the smallest value.
            _, _, node = heapq.heappop(heap)

            # Append this smallest node to the merged linked list.
            current.next = node
            current = current.next

            # If this node has a next node, that next node is now
            # the new candidate from this same sorted list.
            if node.next is not None:
                heapq.heappush(heap, (node.next.val, unique_id, node.next))
                unique_id += 1

        # The merged list starts after the dummy node.
        return dummy.next
```

---

4. Complexity Analysis

Let:

- `N` be the total number of nodes across all linked lists.
- `k` be the number of linked lists.

Time Complexity

```
O(N log k)
```

Explanation:

- At most `k` nodes are stored in the heap at any time.
- Each of the `N` nodes is pushed and popped from the heap once.
- Each heap operation costs `O(log k)`.

Therefore:

```
N nodes × log k heap cost = O(N log k)
```

Space Complexity

```
O(k)
```

Explanation:

- The heap stores at most one node from each list at a time.
- Therefore, the heap size is at most `k`.

The output list reuses the existing nodes, so we do not count extra space for new nodes.

---

5. Alternative Approaches

1. Divide and Conquer

Repeatedly merge lists in pairs, similar to merge sort.

Example:

```
merge list 0 with list 1
merge list 2 with list 3
merge list 4 with list 5
...
repeat until one list remains
```

Time complexity:

```
O(N log k)
```

Space complexity:

```
O(1)
```

or `O(log k)` if implemented recursively.

This is also a very common and efficient solution.

---

2. Merge Lists One by One

Start with an empty result list, then merge each linked list into it one at a time.

Time complexity:

```
O(kN)
```

This is less efficient because the merged list keeps growing, and we repeatedly scan large portions of it.

---

3. Flatten and Sort

Collect all node values into an array, sort the array, then build a new linked list.

Time complexity:

```
O(N log N)
```

Space complexity:

```
O(N)
```

This is simple but does not take full advantage of the fact that each input list is already sorted.

The heap-based solution is usually the most direct and clean approach for this problem because it always chooses the smallest available node among the `k` current candidates.